

Mise à niveau Linux

Les bases pour l'apprenti administrateur système

Dr. Maïssa MBAYE

maïssa.mbaye@ugb.edu.sn

**Master Ingénierie Informatique et Technologies de l'Information et de la
Communication (2ITIC)**



Université Gaston Berger de Saint-Louis (UGB)
Section informatique, UFR SAT

Année académique 2016-2017

Première partie I

Linux de tous les jours

- 1 Système de fichier de Linux
 - Fichier, arborescence et chemin
 - Navigation dans le système de fichier
 - Gestion des fichiers et répertoires
 - Recherche de fichiers
 - Liens : symboliques et physiques
 - Gestion des droits sur les fichiers

- 1 Système de fichier de Linux
 - Fichier, arborescence et chemin
 - Navigation dans le système de fichier
 - Gestion des fichiers et répertoires
 - Recherche de fichiers
 - Liens : symboliques et physiques
 - Gestion des droits sur les fichiers

Notion de fichier : sens commun

Usage commun du terme **fichier** en informatique

Collection de données qui a un nom, généralement stockées sur un support de stockage (disque dur, CD/DVD, Disque Flash, ...) et traitées comme une seule entité.

Exemples :

- Une image est une collection de données (coordonnées-couleurs) qui permettent de dessiner tous les points qui la constitue
- Un document .pdf est une collection d'informations d'affichage (objects et mise en page) permettant à un lecteur de l'afficher correctement
- ...

Les fichiers constituent le composant de base de stockage des données des utilisateurs



Notion de fichier : sens **Unix** du terme

Au sens Unix du terme

Un fichier est :

- toute **source** avec un nom à partir de laquelle des données peuvent être lues (source de input) ;
- ou toute **cible** avec un nom vers laquelle des données peuvent être écrites (cible de output).

"Sur Unix tout est fichier... ou presque..."

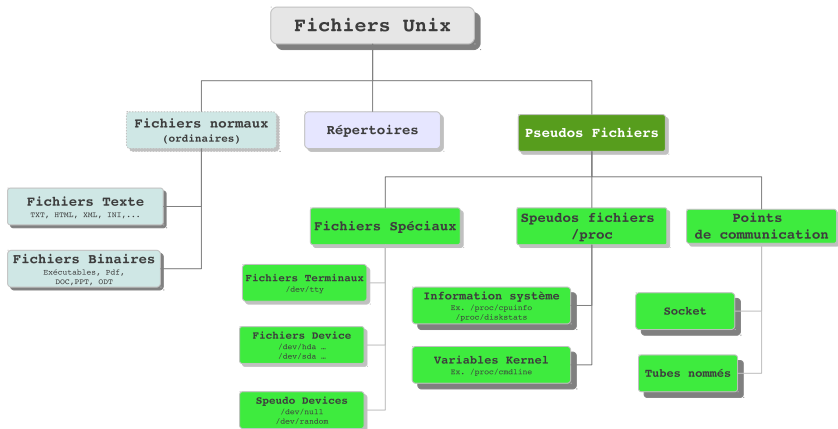
L'intérêt d'une définition aussi large réside en la possibilité de manipuler tous les objets du système avec quelques opérations de base des fichiers : ouverture, fermeture, lecture, écriture...

Exemples de fichiers :

- L'accès aux données provenant du clavier peut se faire comme pour un fichier (source de données lues)
- L'affichage à l'écran peut se faire comme pour l'écriture dans un fichier (cible de données écrites)



Taxonomie des types de fichiers Unix



Affichage du type de fichier : commande file(1)

- Le type des fichiers sous Linux n'est pas déterminé par l'extension mais le magic number (déterminé par son contenu)
- La commande `file` affiche le type de fichier en se basant sur le magic number

Exemple

```
$ file /bin/bash
/bin/bash: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked
(uses shared libs), for GNU/Linux 2.6.24, BuildID[sha1]=0xf199a4a89ac968c2e0e99f2410600b9d7e995187,
stripped
$ file /etc/hosts
/etc/hosts: ASCII text
$ file part1.pdf
part1.pdf: PDF document, version 1.5
$ file /dev/sda2
/dev/sda2: block special
$ file /dev/tty0
/dev/tty0: character special
$ file python-2.7.1-docs-html.tar.bz2
python-2.7.1-docs-html.tar.bz2: bzip2 compressed data, block size = 900k
$ file /etc/
/etc/: directory
```



Affichage du type des fichiers : file(1) suite

Exemples

```
> file convert-to-utf.sh  
convert-to-utf.sh: a bash script text executable
```



Affichage du type des fichiers : file(1) suite

Exemples

```
> file convert-to-utf.sh
convert-to-utf.sh: a bash script text executable

> file networking-mptcp.pdf
networking-mptcp.pdf: PDF document, version 1.4
```



Affichage du type des fichiers : file(1) suite

Exemples

```
> file convert-to-utf.sh
convert-to-utf.sh: a bash script text executable

> file networking-mptcp.pdf
networking-mptcp.pdf: PDF document, version 1.4

> file hello_word_file
hello_word_file: empty
```



Affichage du type des fichiers : file(1) suite

Exemples

```
> file convert-to-utf.sh
convert-to-utf.sh: a bash script text executable

> file networking-mptcp.pdf
networking-mptcp.pdf: PDF document, version 1.4

> file hello_word_file
hello_word_file: empty

> file /usr/bin/firefox
/usr/bin/firefox: symbolic link to '../lib/firefox-6.0/firefox.sh'
```



Affichage du type des fichiers : file(1) suite

Exemples

```
> file convert-to-utf.sh
convert-to-utf.sh: a bash script text executable

> file networking-mptcp.pdf
networking-mptcp.pdf: PDF document, version 1.4

> file hello_word_file
hello_word_file: empty

> file /usr/bin/firefox
/usr/bin/firefox: symbolic link to '../lib/firefox-6.0/firefox.sh'

> file omnetpp
omnetpp: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically
(uses shared libs), for GNU/Linux 2.6.15, not stripped
```



Affichage du type des fichiers : file(1) suite

Exemples

```
> file convert-to-utf.sh
convert-to-utf.sh: a bash script text executable

> file networking-mptcp.pdf
networking-mptcp.pdf: PDF document, version 1.4

> file hello_word_file
hello_word_file: empty

> file /usr/bin/firefox
/usr/bin/firefox: symbolic link to '../lib/firefox-6.0/firefox.sh'

> file omnetpp
omnetpp: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically
(uses shared libs), for GNU/Linux 2.6.15, not stripped

> file file.tgz
file.tgz: gzip compressed data, from Unix, last modified: Tue Apr 12 10:01:30
2011
```



Fichiers ordinaires

- Les fichiers ordinaires, aussi appelés fichiers réguliers (*regular*), contiennent des données et sont stockés sur des supports de stockage (disque durs, CD/DVD, carte mémoire, ...)
- Il s'agit du type de fichier le plus utilisé dans la vie de tous les jours par les utilisateurs

Il existe deux types de fichiers ordinaires

- **Les fichiers texte** : contiennent des lignes composées uniquement des caractères imprimables (lettres, chiffres, symboles de ponctuation, tabulations, espaces) et terminées par le caractère nouvelle ligne `\n`
 - Exemple : fichier texte brut, script shell, sources de programmes C, fichiers de configuration, fichier HTML, ...
- **Les fichiers binaires** : contiennent des données non textuelles (binaires) qui n'ont du sens que lorsque le fichier est exécuté ou interprété par un programme
 - Exemple de fichiers binaires : fichiers exécutables, fichiers objets, images, vidéos, fichier doc, tableurs, bases de données, ...



Speudos fichiers

- Les pseudos fichiers ne sont pas utilisés pour stocker des données et n'occupent pas réellement un espace disque.
- Ils sont par contre présents sur le système de fichier pour permettre leur accès comme pour les fichiers normaux

Catégories de pseudos fichiers

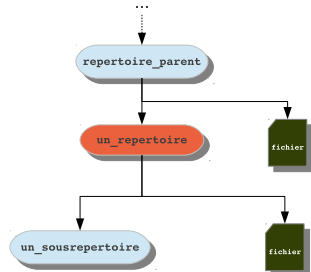
- Fichiers spéciaux ou fichier device :
 - Chaque équipement physique a une représentation sous forme de fichier qui permet d'y accéder
 - Exemple : la souris est accessible sur `/dev/input/mouse0`
- Tubes nommés et sockets :
 - Ils permettent la communication entre deux processus sur le même ordinateur ou entre deux ordinateurs reliés par un réseau
- Fichiers PROC :
 - Ce sont des pseudos fichiers permettant d'accéder aux informations du Kernel de Linux



Fichiers repertoires

Les repertoires sont des conteneurs qui peuvent "contenir" d'autres fichiers (y compris des repertoires)

- Les repertoires ne contiennent pas de données mais permettent d'organiser les fichiers et d'y accéder
 - Les repertoires sont des classeurs de fichiers
- Dans les environnements graphiques le terme dossier (folder) est utilisé
- Un répertoire peut contenir des repertoires → permet une organisation des fichiers en arbre



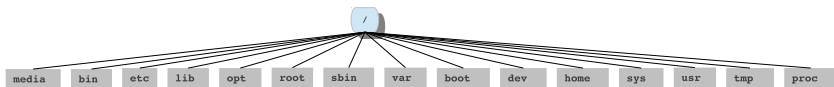
Arborescence du système de fichier de Linux

Un système Linux de base contient plus de 100 000 fichiers réparties dans des répertoires et des sous-répertoires organisés en arbre

- L'ensemble des fichiers et répertoires du système Linux constitue le système de fichier
- Le répertoire spécial qui contient tous les autres est appelé "Root" (Prononcer 'route') et noté /

Liste des sous-répertoires de la racine de Ubuntu 11.04

```
$ ls /
bin boot dev etc home lib media mnt opt proc root sbin sys tmp usr var
```



Arborescence du système de fichier de Linux

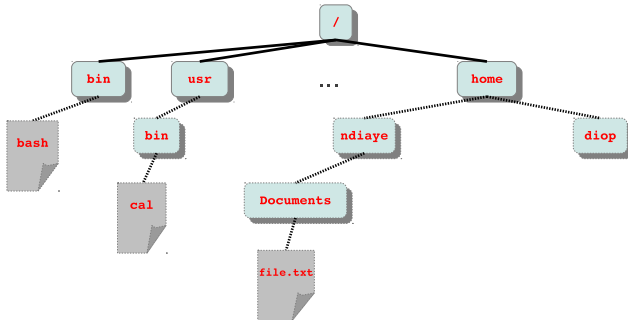
Rôle de quelques répertoires de l'arborescence de **Linux** :

- /** : Racine du système
- /bin** : contient les commandes de bases (grep, mount, cp ...).
- /dev** : contient les fichiers spéciaux correspondant aux périphériques
- /etc** : contient la plupart des fichiers de configuration.
- /home** : contient les répertoires personnels des utilisateurs
- /lib** : contient les principales bibliothèques partagées
- /root** : le répertoire personnel et privé de root.
- /sbin** : Commandes de base nécessaires à l'administration système
- /tmp** : Répertoire temporaire. Contenu susceptible d'être supprimé à chaque redémarrage
- /usr** : Répertoire d'installation des logiciels
- /var** : des données fréquemment réécrites, comme les logs.
- /mnt** : ou **/media** montage de périphériques de stockage (clés USB ou autres)



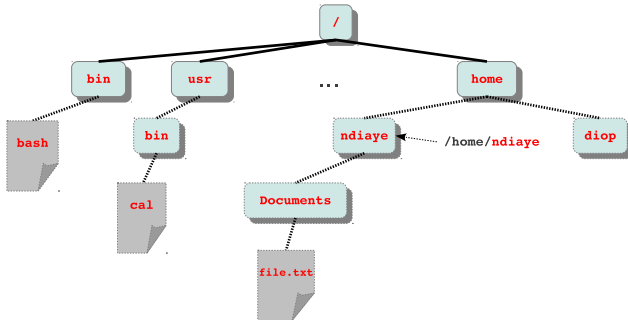
Nom de fichier et chemin

Les fichiers sont localisés dans le système de fichier grâce à leur **chemin** : la liste de tous les répertoires et sous-répertoires qui permettent d'y accéder



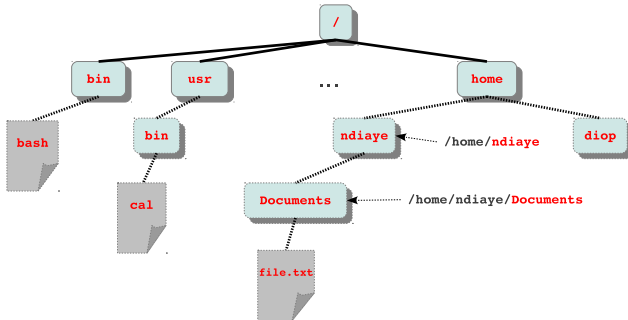
Nom de fichier et chemin

Les fichiers sont localisés dans le système de fichier grâce à leur **chemin** : la liste de tous les répertoires et sous-répertoires qui permettent d'y accéder



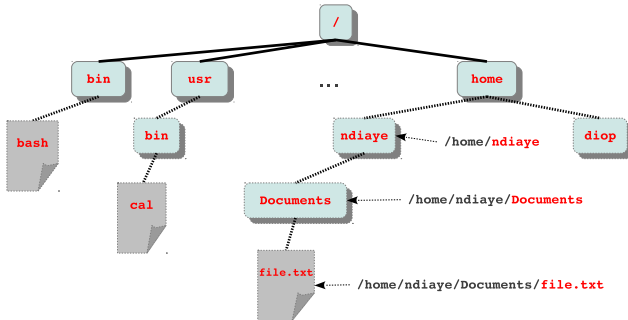
Nom de fichier et chemin

Les fichiers sont localisés dans le système de fichier grâce à leur **chemin** : la liste de tous les répertoires et sous-répertoires qui permettent d'y accéder



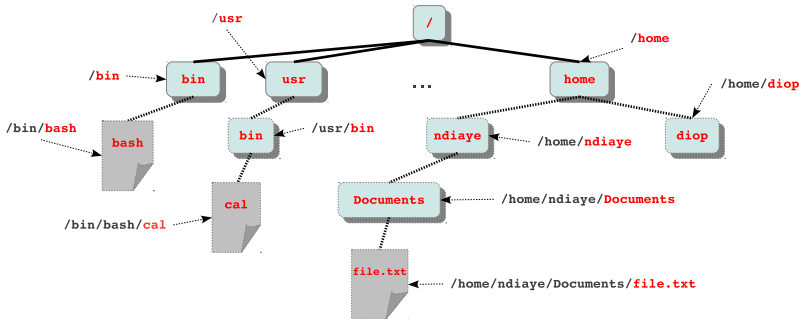
Nom de fichier et chemin

Les fichiers sont localisés dans le système de fichier grâce à leur **chemin** : la liste de tous les répertoires et sous-répertoires qui permettent d'y accéder



Nom de fichier et chemin

Les fichiers sont localisés dans le système de fichier grâce à leur **chemin** : la liste de tous les répertoires et sous-répertoires qui permettent d'y accéder



Chemin absolu, chemin relatif

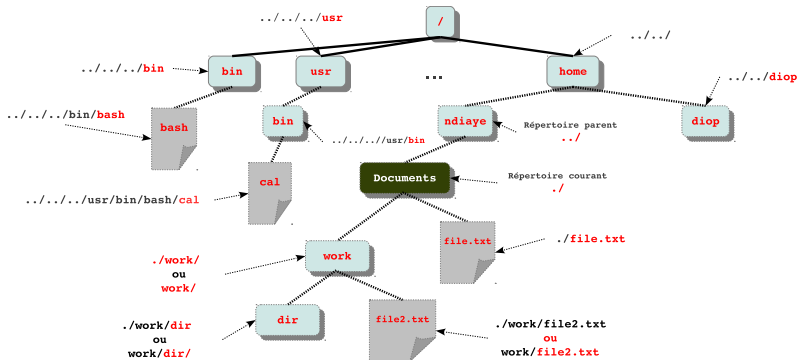
- Le Shell est positionné à tout moment dans un repertoire appelé **repertoire courant**
 - Le repertoire courant est accessible par le repertoire "." (point)
 - Le repertoire parent du repertoire courant est accessible par le repertoire ".." (point-point)
 - Le repertoire parent du repertoire parent du repertoire courant est accessible par le repertoire "../.."
- Tous les autres fichiers peuvent être référencés relativement au repertoire courant

- On parle de **chemin absolu** lorsque les chemins sont référencés par la liste des repertoires et sous repertoires qui permettent d'y accéder depuis la racine
- On parle de **chemin relatif** lorsque le chemin est exprimé relativement au repertoire courant (./, ../, ../..)



Chemin absolu, chemin relatif

Exemple de chemins relatifs



- 1 Système de fichier de Linux
 - Fichier, arborescence et chemin
 - **Navigation dans le système de fichier**
 - Gestion des fichiers et répertoires
 - Recherche de fichiers
 - Liens : symboliques et physiques
 - Gestion des droits sur les fichiers

Chemin et nom de fichier : `dirname`, `basename`, `pwd`

- **`dirname`** : permet de récupérer le répertoire dans le chemin complet d'un fichier
- **`dirname`** : récupère au contraire le nom du fichier
- **`pwd`** (print working directory) : affiche le répertoire courant

Exemples :

```
> dirname /usr/include/linux/socket.h
/usr/include/linux
> basename /usr/include/linux/socket.h
socket.h
> pwd
/home/toto
```



Listage de repertoire : “ls” et ses variantes

```
ls [options] [chemin]
```

ls : liste le contenu d'un repertoire

Options communes de ls :

-a, --all : Affiche les fichiers cachés

-l : Affichage au format long (presque tous les champs d'inode)

-h, --human-readable : Affiche les tailles des fichiers de manière adaptée (K pour Kilobyte, M pour Megabyte et G pour Gigabyte)

-la, -lh, -R autres options communes

Commandes similaires :

dir : Equivalent à 'ls -C -b'

vdir : Equivalent à 'ls -l -b'

dircolor : pour mettre des couleurs sur la sortie



Listage de repertoire : “ls” et ses variantes

Exemple de sortie terminal

```
> ls media/  
Data OS  
> ls -l /media/  
total 28  
drwx--- 1 mbyte mbyte 16384 2011-08-17 18:51 Data  
drwx--- 1 mbyte mbyte 12288 2011-08-19 16:33 OS  
> ls -lh media/  
total 28K  
drwx--- 1 mbyte mbyte 16K 2011-08-17 18:51 Data  
drwx--- 1 mbyte mbyte 12K 2011-08-19 16:33 OS  
> ls -a /media/  
. .. Data OS
```



Listage de repertoire : “ls” et ses variantes

Exemple de sortie terminal

```
> ls media/  
Data OS  
> ls -l /media/  
total 28  
drwx--- 1 mbyte mbyte 16384 2011-08-17 18:51 Data  
drwx--- 1 mbyte mbyte 12288 2011-08-19 16:33 OS  
> ls -lh media/  
total 28K  
drwx--- 1 mbyte mbyte 16K 2011-08-17 18:51 Data  
drwx--- 1 mbyte mbyte 12K 2011-08-19 16:33 OS  
> ls -a /media/  
. .. Data OS
```



Listage de repertoire : “ls” et ses variantes

Exemple de sortie terminal

```
> ls media/
Data OS
> ls -l /media/
total 28
drwx--- 1 mbyte mbyte 16384 2011-08-17 18:51 Data
drwx--- 1 mbyte mbyte 12288 2011-08-19 16:33 OS
> ls -lh media/
total 28K
drwx--- 1 mbyte mbyte 16K 2011-08-17 18:51 Data
drwx--- 1 mbyte mbyte 12K 2011-08-19 16:33 OS
> ls -a /media/
. .. Data OS
```



Listage de repertoire : “ls” et ses variantes

Exemple de sortie terminal

```
> ls media/
Data OS
> ls -l /media/
total 28
drwx--- 1 mbyte mbyte 16384 2011-08-17 18:51 Data
drwx--- 1 mbyte mbyte 12288 2011-08-19 16:33 OS
> ls -lh media/
total 28K
drwx--- 1 mbyte mbyte 16K 2011-08-17 18:51 Data
drwx--- 1 mbyte mbyte 12K 2011-08-19 16:33 OS
> ls -a /media/
. .. Data OS
```



Listage de repertoire : “ls” et ses variantes

la sortie de `ls` associe à chaque type de fichier une lettre

- : Les fichiers ordinaires (tous les fichiers courants, .txt, .doc, .mp3 .jpeg ou sans extensions)
- `c` ou `b` : Les fichiers spéciaux (fichiers associés aux périphériques, localisés dans le repertoire `/dev/`)
- `d` : Les répertoires
- `l` : Les liens, un lien permet d'accéder à un même fichier ou répertoire en utilisant un nom différent.



Navigation dans le système de fichier : "cd"

cd : Changedir

la commande `cd` (change directory) permet de changer le repertoire de travail du Shell, de naviguer dans le système de fichier.

```
> cd /  
> pwd  
> cd ~  
> pwd  
/home/toto  
> cd public; pwd  
> cd -  
/home/toto  
> cd ..; pwd  
/home  
> cd .; pwd  
/home
```



Afficher l'arborescence : commande tree(1)

La commande `tree(1)` permet d'afficher la structure arborescente d'un repertoire (sous-repertoires et fichiers ordinaires)

Exemple :

```
mbaye@mbaye-U36JC: ~/Bureau/tmp
mbaye@mbaye-U36JC:~/Bureau/tmp$ tree ./
./
├── p1
│   └── f1.txt
├── p2
│   ├── p2-1
│   │   └── f2.txt
│   ├── p2-2
│   └── p2-3
├── p3
│   ├── p3-1
│   │   └── p3-1-1
│   ├── p3-2
│   └── p3-3
├── p4
└── p5

12 directories, 2 files
mbaye@mbaye-U36JC:~/Bureau/tmp$
```



- 1 Système de fichier de Linux
 - Fichier, arborescence et chemin
 - Navigation dans le système de fichier
 - **Gestion des fichiers et répertoires**
 - Recherche de fichiers
 - Liens : symboliques et physiques
 - Gestion des droits sur les fichiers

Création de répertoires : mkdir(1)

Syntaxe simplifiée

```
mkdir [options] nom_repertoire_1 nom_repertoire_2 ... nom_repertoire_n
```

- Création d'un répertoire (chemin relatif)

```
> mkdir un-repertoire un-autre-repertoire un-troisieme-repertoire
```



Création de répertoires : mkdir(1)

Syntaxe simplifiée

```
mkdir [options] nom_repertoire_1 nom_repertoire_2 ... nom_repertoire_n
```

- Création d'un répertoire (chemin relatif)
 - > `mkdir un-repertoire un-autre-repertoire un-troisieme-repertoire`
- Création d'un répertoire avec un chemin complet
 - > `mkdir /home/toto/Documents/un-repertoire`



Création de répertoires : mkdir(1)

Syntaxe simplifiée

```
mkdir [options] nom_repertoire_1 nom_repertoire_2 ... nom_repertoire_n
```

- Création d'un répertoire (chemin relatif)
> `mkdir un-repertoire un-autre-repertoire un-troisieme-repertoire`
- Création d'un répertoire avec un chemin complet
> `mkdir /home/toto/Documentes/un-repertoire`
- Création d'un répertoire et créer les répertoires parent s'ils n'existent pas
> `mkdir -p /home/toto/Documentes/ancetre/grand-pere/pere/un-repertoire`



Création de répertoires : mkdir(1)

Syntaxe simplifiée

```
mkdir [options] nom_repertoire_1 nom_repertoire_2 ... nom_repertoire_n
```

- Création d'un répertoire (chemin relatif)
> `mkdir un-repertoire un-autre-repertoire un-troisieme-repertoire`
- Création d'un répertoire avec un chemin complet
> `mkdir /home/toto/Documentes/un-repertoire`
- Création d'un répertoire et créer les répertoires parent s'ils n'existent pas
> `mkdir -p /home/toto/Documentes/ancetre/grand-pere/pere/un-repertoire`
- Création d'un répertoire avec spécification des droits
> `mkdir -m 755 repertoire`



Renommage/Déplacement de fichiers et répertoire : `mv`

La commande `mv` permet

- de déplacer un fichier
- de renommer un fichier (sous Linux renommer un fichier se fait par son déplacement sous un autre nom dans le même répertoire)

Syntaxe

Déplacement de fichier

```
> mv file_1 file_2 ... repertoire-destination
```

Renommage de fichiers

```
> mv nom-originale-de-fichier nouveau-nom
```



Renommage/Déplacement de fichiers et répertoire : mv

Exemples

```
> ls
```

```
unautrerepertoire  unfichier.txt  unrepertoiredestination
```



Renommage/Déplacement de fichiers et répertoire : mv

Exemples

```
> ls
```

```
unautrerepertoire  unfichier.txt  unrepertoiredestination
```

```
> mv unfichier.txt unrepertoiredestination/
```



Renommage/Déplacement de fichiers et répertoire : mv

Exemples

```
> ls
```

```
unautrerepertoire  unfichier.txt  unrepertoiredestination
```

```
> mv unfichier.txt unrepertoiredestination/
```

```
> ls
```

```
unautrerepertoire  unrepertoiredestination
```



Renommage/Déplacement de fichiers et répertoire : mv

Exemples

```
> ls  
unautrerepertoire  unfichier.txt  unrepertoiredestination
```

```
> mv unfichier.txt unrepertoiredestination/
```

```
> ls  
unautrerepertoire  unrepertoiredestination
```

```
> ls unrepertoiredestination/  
unfichier.txt
```



Renommage/Déplacement de fichiers et répertoire : mv

Exemples

```
> ls
```

```
unautrerepertoire  unfichier.txt  unrepertoiredestination
```

```
> mv unfichier.txt unrepertoiredestination/
```

```
> ls
```

```
unautrerepertoire  unrepertoiredestination
```

```
> ls unrepertoiredestination/
```

```
unfichier.txt
```

```
> mv unautrerepertoire/ unautrenomderepertoire
```



Renommage/Déplacement de fichiers et répertoire : mv

Exemples

```
> ls
```

```
unautrerepertoire  unfichier.txt  unrepertoiredestination
```

```
> mv unfichier.txt unrepertoiredestination/
```

```
> ls
```

```
unautrerepertoire  unrepertoiredestination
```

```
> ls unrepertoiredestination/
```

```
unfichier.txt
```

```
> mv unautrerepertoire/ unautrenomderepertoire
```

```
> ls
```

```
unautrenomderepertoire  unrepertoiredestination
```



Suppression de fichiers et répertoires : `rm`

La commande `rm(remove)` permet de supprimer des fichiers ou répertoires

Suppression de fichier ou répertoire vide

```
> rm [-i] nom-fichier-ou-répertoire-vide  
> rmdir nom-répertoire-vide
```

Suppression récursive de répertoire non vide

```
> rm [-i] -rf nom-répertoire  
> rmdir --ignore-fail-on-non-empty nom-répertoire
```

Pour avoir plus d'infos et d'exemples

```
> info coreutils 'rm invocation'
```

La commande `shred` permet une suppression sécurisée et définitive d'un fichier



Copie de fichiers et répertoires : cp(1)

Syntaxe

```
cp source destination
```

Quelques options intéressantes :

- -a --archive : copie avec preservation des attributs des fichiers et répertoires
- -i, --interactive : demande avant d'écraser la destination
- -R, -r, --recursive : copie récursive (de fichiers et répertoires)
- -u, --update : Copie seulement si le nouveau fichier est plus récent que la destination
- -v, --verbose : Affiche ce qui est en cours de copie



Copie de fichiers et répertoires : cp(1)

Exemples :

- Copie interactive de fichier (verbose)

```
> cp -iv nom_fichier_original nom_fichier_destination
```

- Copie complète d'un repertoire (écrase les fichiers sans demander)

```
> cp -var repertoire nom_destination
```

Pour avoir plus d'infos et d'exemples

```
> info coreutils 'cp invocation'
```



Création de fichiers : touch

La commande touch permet de :

- créer un fichier vide

```
> touch hello_word_file
> ls -l hello_word_file
-rw-r-r- 1 toto toto 0 2011-08-24 18:24 hello_word_file
```

- ou de mettre à jour la date du dernier accès et de la dernière modification

```
> ls -l hello_word_file
-rw-r-r- 1 toto toto 0 2011-08-24 18:24 hello_word_file
> touch hello_word_file
> ls -l hello_word_file
-rw-r-r- 1 toto toto 0 2011-08-24 18:27 hello_word_file
```



- 1 **Système de fichier de Linux**
 - Fichier, arborescence et chemin
 - Navigation dans le système de fichier
 - Gestion des fichiers et répertoires
 - **Recherche de fichiers**
 - Liens : symboliques et physiques
 - Gestion des droits sur les fichiers

Recherche de fichiers avec : find

La commande **find** permet de rechercher un fichier selon plusieurs critères

- Nom, type de fichier
- Taille
- UID,GID
- Permissions d'accès
- Heure d'accès
- Type de système de fichiers

Syntaxe de find

```
find [options]... [<search-root>] <search-expression>
```

↑
-H, -L, -P

↑
Répertoires

↑
Mini langage de filtrage



Recherche de fichiers avec : find

- Recherche d'un fichier par le nom¹

```
> find repertoire -name myfile.txt -print
> find . -name "*.txt" -print
> find "$HOME/" -name myfile.txt -print
#respecter la casse
> find . -iname "error" -print
```

- Recherche selon la permission

```
> find . -perm 777 -print
> find . -perm -o=w
```

- Recherche selon la taille du fichier

```
> find /home -name "*.txt" -size 100k
> find /home -name "*.txt" -size +100k
> find /home -name "*.txt" -size -100k
```

1. Attention les options de find sont souvent de la catégorie ambiguë avec un tiret suivi de mot



Recherche de fichiers avec : find

- Recherche d'un type de fichier

```
> find . -type d -print
> find . -type f -print
> find . -type l -print
```

- Controle de la profondeur de recherche

```
> find . -maxdepth 1 -type f -newer first_file
```

- Recherche selon la date de la dernière modification

```
> find . -mtime 0 #fichiers modifiés entre maintenant et
les dernière 24H > find . -mtime -1 #il y a moins de 24H
> find . -mtime 1 #entre 24H et 48H
> find . -mtime +1 #il y a plus de 24H
> find . -mmin +5 -mmin -10 #Entre 6mn et 9mn
```



Recherche de fichier avec locate

- La commande `locate` permet de faire une recherche rapide d'un fichier en se basant sur le nom
- La commande se base sur une base de données des fichiers présents sur le système de fichier (BD mise à jour par `updatedb`)

synthaxe :

```
locate [options] [repertoire/fichier]
```

Exemple :

```
$ locate hosts  
/etc/avahi/hosts  
/etc/hosts  
/usr/share/hosts
```

Si la base de données de `locate` n'est pas à jour, l'exécution de la commande `updatedb` permet de la forcer



Recherche d'une commande which

- La commande `which` permet de retrouver le chemin complet d'une commande
- Très utile lorsque plusieurs versions d'une commande sont installées

syntaxe :

```
which [-a] nom_de_fichier...
```

Exemples :

```
> which man
/usr/bin/man
> which lspci
/usr/bin/lspci
> which grep
/bin/grep
> which ifconfig
/sbin/ifconfig
> which vlc
/usr/local/bin/vlc
```



Recherche avec la commande whereis

La commande whereis affiche

- Le chemin de l'exécutable d'un binaire,
- Son archive de page manuelle
- Le code source de la commande (s'il est installé)

synthaxe : `whereis [OPTIONS] [COMMANDE/FICHIER]` Exemple :

```
$ whereis ls
```

```
ls: /bin/ls /usr/share/man/man1/ls.1.gz
```

Usages courants :

```
$ whereis [COMMANDE]    # affiche le chemin de la commande
$ whereis -b [COMMANDE]  # affiche le chemin du fichier binaire seulement
$ whereis -m [COMMANDE]  # affiche la page manuelle seulement
$ whereis -s [COMMANDE]  # affiche le code source seulement
```



- 1 Système de fichier de Linux
 - Fichier, arborescence et chemin
 - Navigation dans le système de fichier
 - Gestion des fichiers et répertoires
 - Recherche de fichiers
 - **Liens : symboliques et physiques**
 - Gestion des droits sur les fichiers

Liens symboliques et liens physiques

Liens

Les liens sont des copies virtuelles des fichiers physiques.

Deux catégories de liens :

- lien symbolique ou soft : un lien vers le fichier physique
- lien physique : une nouvelle référence sur les données



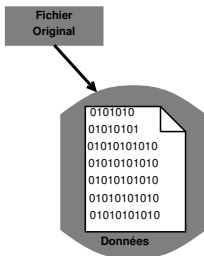
Liens symboliques et liens physiques

Liens

Les liens sont des copies virtuelles des fichiers physiques.

Deux catégories de liens :

- lien symbolique ou soft : un lien vers le fichier physique
- lien physique : une nouvelle référence sur les données



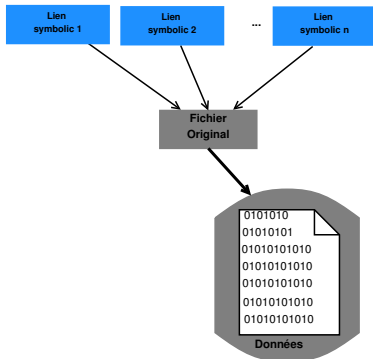
Liens symboliques et liens physiques

Liens

Les liens sont des copies virtuelles des fichiers physiques.

Deux catégories de liens :

- lien symbolique ou soft : un lien vers le fichier physique
- lien physique : une nouvelle référence sur les données



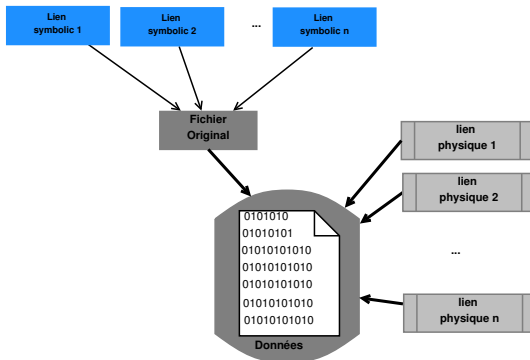
Liens symboliques et liens physiques

Liens

Les liens sont des copies virtuelles des fichiers physiques.

Deux catégories de liens :

- lien symbolique ou soft : un lien vers le fichier physique
- lien physique : une nouvelle référence sur les données



Création de liens : ln(1)

La création de liens peut se faire avec les commandes : cp et ln

Création de liens symboliques :

```
> cp -s nom_fichier_original nom_lien_destination  
> ln -s nom_fichier_original nom_lien_destination
```

Création de liens physiques :

```
> cp -l nom_fichier_original nom_lien_destination  
> ln nom_fichier_original nom_lien_destination
```

Pour avoir plus d'infos et d'exemples

```
> info coreutils 'ln invocation'
```



Création de liens : ln(1)

Exemples

```
> ls -lk
total 4,0K
drwxr-xr-x 2 mbye 4,0K 2011-08-25 20:28 original_file
-rw-r-r- 1 mbye 12 2011-08-25 20:50 unfichier.txt
```



Création de liens : ln(1)

Exemples

```
> ls -lk
total 4,0K
drwxr-xr-x 2 mbye 4,0K 2011-08-25 20:28 original_file
-rw-r-r- 1 mbye 12 2011-08-25 20:50 unfichier.txt

> ln -s original_file lien_sybmolique_file
> ls -oh
total 4,0K
lrwxrwxrwx 1 mbye 13 2011-08-25 20:45 lien_sybmolique_file -> original_file
drwxr-xr-x 2 mbye 4,0K 2011-08-25 20:43 original_file
-rw-r-r- 1 mbye 12 2011-08-25 20:50 unfichier.txt
```



Création de liens : ln(1)

Exemples

```
> ls -lk
total 4,0K
drwxr-xr-x 2 mbye 4,0K 2011-08-25 20:28 original_file
-rw-r-r- 1 mbye 12 2011-08-25 20:50 unfichier.txt

> ln -s original_file lien_sybmolique_file
> ls -oh
total 4,0K
lrwxrwxrwx 1 mbye 13 2011-08-25 20:45 lien_sybmolique_file -> original_file
drwxr-xr-x 2 mbye 4,0K 2011-08-25 20:43 original_file
-rw-r-r- 1 mbye 12 2011-08-25 20:50 unfichier.txt

> ln unfichier.txt unlien-physique.txt
> ln -P unfichier.txt un-autre-lien-physique.txt
```



Création de liens : ln(1)

Exemples

```
> ls -lk
total 4,0K
drwxr-xr-x 2 mbye 4,0K 2011-08-25 20:28 original_file
-rw-r-r- 1 mbye 12 2011-08-25 20:50 unfichier.txt

> ln -s original_file lien_sybomlique_file
> ls -oh
total 4,0K
lrwxrwxrwx 1 mbye 13 2011-08-25 20:45 lien_sybomlique_file -> original_file
drwxr-xr-x 2 mbye 4,0K 2011-08-25 20:43 original_file
-rw-r-r- 1 mbye 12 2011-08-25 20:50 unfichier.txt

> ln unfichier.txt unlien-physique.txt
> ln -P unfichier.txt un-autre-lien-physique.txt

> ls -li
4076163 lien_sybomlique_file      4076388 original_file
4076211 un-autre-lien-physique.txt 4076211 unfichier.txt
4076211 unlien-physique.txt
```



Affichage des statistiques des fichiers : stat(1)

la commande stat permet d'afficher toutes les informations contenues dans l'*i-node* du fichier.



Affichage des statistiques des fichiers : stat(1)

la commande stat permet d'afficher toutes les informations contenues dans l'*i-node* du fichier.

Exemples

```
> stat Bureau/
  File: «Bureau/»
  Size: 4096      Blocks: 8          IO Block: 4096   répertoire
Device: 804h/2052d Inode: 4063235    Links: 5
Access: (0755/drwxr-xr-x)  Uid: ( 1000/   mbaye)   Gid: ( 1000/   mbaye)
Access: 2011-08-26 20:00:54.547058018 +0000
Modify: 2011-08-25 18:32:56.355880259 +0000
Change: 2011-08-25 18:32:56.355880259 +0000
```



Affichage des statistiques des fichiers : stat(1)

la commande stat permet d'afficher toutes les informations contenues dans l'*i-node* du fichier.

Exemples

```
> stat Bureau/
  File: «Bureau/»
  Size: 4096      Blocks: 8          IO Block: 4096   répertoire
Device: 804h/2052d Inode: 4063235    Links: 5
Access: (0755/drwxr-xr-x)  Uid: ( 1000/   mbyte)   Gid: ( 1000/   mbyte)
Access: 2011-08-26 20:00:54.547058018 +0000
Modify: 2011-08-25 18:32:56.355880259 +0000
Change: 2011-08-25 18:32:56.355880259 +0000

> stat convert-to-utf.sh
  File: «convert-to-utf.sh»
  Size: 589      Blocks: 8          IO Block: 4096   fichier
Device: 804h/2052d Inode: 4076463    Links: 1
Access: (0755/-rwxr-xr-x)  Uid: ( 1000/   mbyte)   Gid: ( 1000/   mbyte)
Access: 2011-08-26 17:32:22.479666445 +0000
Modify: 2011-08-26 17:32:02.779666440 +0000
Change: 2011-08-26 17:32:03.187666437 +0000
```



- 1 Système de fichier de Linux
 - Fichier, arborescence et chemin
 - Navigation dans le système de fichier
 - Gestion des fichiers et répertoires
 - Recherche de fichiers
 - Liens : symboliques et physiques
 - Gestion des droits sur les fichiers

Permissions des fichiers

Principe

- Chaque fichier appartient à un utilisateur et à un ou plusieurs groupes
- Chaque utilisateur peut avoir des droits (permissions) de **lecture**, **écriture** et d' **execution** sur un fichier
- Chaque fichier définit des permissions pour trois classes d'utilisateurs :
 - **(u)** Utilisateur propriétaire (créateur du fichier)
 - **(g)** Groupe propriétaire
 - **(o)** Les autres utilisateurs



Permissions des fichiers



Permissions des fichiers

```
$~> ls -lh file
```

```
-rwxrwxrwx 1 modlo grpmodlo 589 2011-08-26 17:32 file
```



-rwxrwxrwx



Permissions des fichiers

```
$~> ls -lh file
```

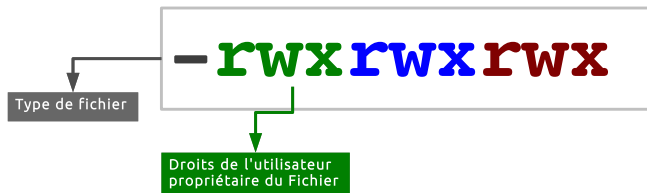
```
-rwxrwxrwx 1 modlo grpmodlo 589 2011-08-26 17:32 file
```



Permissions des fichiers

```
$~> ls -lh file
```

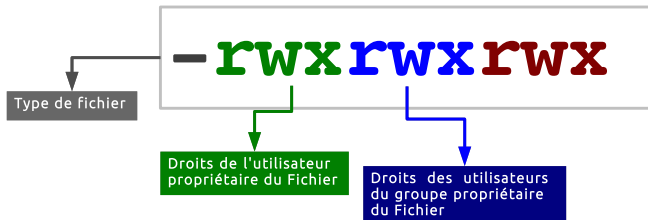
```
-rwxrwxrwx 1 modlo grpmodlo 589 2011-08-26 17:32 file
```



Permissions des fichiers

```
$~> ls -lh file
```

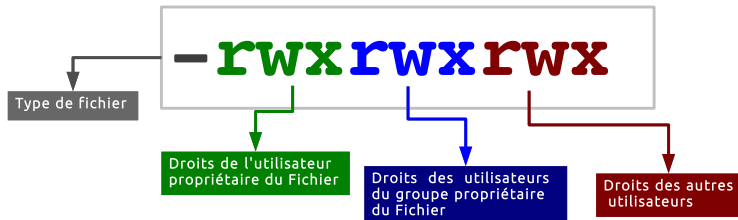
```
-rwxrwxrwx 1 modlo grpmodlo 589 2011-08-26 17:32 file
```



Permissions des fichiers

```
$~> ls -lh file
```

```
-rwxrwxrwx 1 modlo grpmodlo 589 2011-08-26 17:32 file
```



Décodage des permissions des fichiers

Les permissions des chaque classe d'utilisateur sont définit par trois flags(bits) :

	Significations sur les fichiers simples	Signification sur les répertoires
r	Droit de lecture du contenu du fichier	Droit de voir tous les fichiers présent dans le répertoire
w	Droit d'écriture dans le fichier (Modification du contenu du fichier)	Droit de créer, supprimer, renommer les fichiers dans le répertoire
x	Droits d'exécution: détermine si le fichier est un exécutable (programme/script exécutable) ou non	Autorisation d'utiliser le nom du répertoire pour accéder à un fichier dans le répertoire

Flag	Valeur binaire (décimale)	Signification
rwX	$(111)_2 = (7)_{10}$	Tous les droits (lecture écriture et exécution), droit du propriétaire
rw-	$(110)_2 = (6)_{10}$	Lecture / Écriture
r-X	$(101)_2 = (5)_{10}$	Lecture/Exécution
---	$(000)_2 = (0)_{10}$	Aucun droit (Protection maximale)
r--	$(100)_2 = (4)_{10}$	Lecture seule (Protection contre la modification)
...	...	...



Décodage des permissions des fichiers

Exercice : Quelle est la signification de ces droits ?

```
> ls -lh /usr/bin/lspci  
-rwxr-xr-x 1 root root 64K 2011-03-17 13:27 /usr/bin/lspci
```



Décodage des permissions des fichiers

Exercice : Quelle est la signification de ces droits ?

```
> ls -lh /usr/bin/lspci  
-rwxr-xr-x 1 root root 64K 2011-03-17 13:27 /usr/bin/lspci
```

Réponse

- **rwx** (tous droits) pour le propriétaire (root)
- **r-x** (lecture/exécution) le groupe propriétaire
- **r-x** pour tous les autres



Affichage des droits avec la commande getacl

Les commandes getacl/setacl permettent de manipuler les droits d'accès sur les fichiers

```
$ getfacl p1
# file: p1
# owner: mbaye
# group: mbaye
user::rwx
group::rwx
other::r-x
```



Modification des permissions : chmod(1)

Syntaxe

```
chmod [options] [mode-symbolic | mode-octal] fichier
```

- Options : (-c, -R, -v, ...)
- Mode-symbolic : valeur symbolic des permissions
- Mode-octal : Valeur octale des nouveaux droits (755,700,...)



Utilisation des permissions symboliques

Forme générale

[ugoa...][+ -=][rwx...]

- **u** (Utilisateur[user]), **g** (groupe[groupe]), **o** (les autres [other]), **a** (tous [all])
- **+** (Ajouter des droits), **-** soustraire des droits, **=** (mettre des droits exacts)
- **r,w,x** (classique)...

Exemples :

```
> ls -lh script.sh
-rwxrwxrwx 1 mbye mbye 7 2011-09-01 22:21 script.sh
> chmod -v a-r script.sh
le mode de «script.sh» a été modifié en 0333 (-wx-wx-wx).
> ls -lh script.sh
--wx-wx-wx 1 mbye mbye 7 2011-09-01 22:21 script.sh
> chmod -v u+r script.sh
le mode de «script.sh» a été modifié en 0733 (rwx-wx-wx).
> chmod -v o-rwx script.sh
le mode de «script.sh» a été modifié en 0730 (rwx-wx--).
> chmod -v g=rx script.sh
le mode de «script.sh» a été modifié en 0750 (rwxr-x--).
> chmod -v u+rwx,g=rx,o=x script.sh
le mode de «script.sh» a été modifié en 0751 (rwxr-x-x).
```



Utilisation des permissions octales

Il s'agit d'une forme résumée des droits des trois classes d'utilisateurs

Forme générale

XYZ

- X valeur octale des droits de l'utilisateur
- Y valeur octale des droits du groupe
- Z valeur octale des droits des autres

Exemples :

```
> chmod -R -v 755 repertoire/
le mode de «repertoire/» a été conservé à 0755 (rwxr-xr-x).
```

```
> chmod -v 600 script.sh
le mode de «script.sh» a été modifié en 0600 (rw----).
```

```
> chmod -v 644 script.sh
le mode de «script.sh» a été modifié en 0644 (rw-r-r-).
```



Permissions de fichiers remarquables

Les droits les plus souvent utilisés sont résumés dans cet tableau :

Droit	Commentaire
755	Les permissions généralement préférées pour tous les fichiers sur le disque
700	Données privées
500	Données extrêmement privées et que vous n'avez pas envie de supprimer par accident (protection contre l'écriture)
775	Généralement utilisé dans les fichiers et répertoires partagés dans le groupes pour travailler
770	Fichiers privés au sein du groupe
750	Données consultable seulement par les éléments du groupes sans possibilité de modification
777	Quelque chose à ne jamais faire !!! sauf si vous savez ce que vous faites



Propriétaire et groupe d'un fichier : `chown`, `chgrp`

- la commande `chown` permet de changer le propriétaire et/ou le groupe propriétaire d'un fichier
- la commande `chgrp` permet de changer le groupe par défaut propriétaire d'un fichier

Syntaxes

```
> chown user filename
> chown user:group filename
> chown :group filename

> chgrp group filename
```

Remarque : d'anciennes versions utilisent "." (point) comme séparateur à la place du ":" (deux points), mais la version GNU de la commande supporte les deux



Propriétaire et groupe d'un fichier : chown, chgrp

Exemples

```
> chown -v carlas:share index.txt
changed ownership of 'index.txt' to carlas:share

> chown -v :share toc.txt
changed ownership of 'toc.txt' to :share

> chown carlas file.txt file2.txt file3.txt
> chown carlas *.txt

> chown -R -v piglet /shared/scripts
changed ownership of 'scripts' to piglet
changed ownership of 'scripts/backups.tgz' to piglet
changed ownership of 'scripts/fake-spec-rpm' to piglet

> chgrp -R -v www-data ~/pubic_html
```



Droits par défaut des fichiers : umask

Exercice

Essayez de découvrir l'utilisation de cette commande



Bibliographie



Carla Schroder

Linux Cookbook : Practical Advice for Linux System Administrators
O'Reilly, November 2004



Richard Blum,

Linux Command Line and Shell Scripting Bible,
Wiley Publishing, Inc. 2008



Michael Palmer

Guide to UNIX Using Linux, FOURTH EDITION
Course Technology, Cengage Learning, 2008



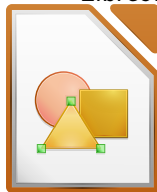
Credits

Cette présentation a été réalisée avec des outils libres :

la classe BEAMER de $\text{\LaTeX}$



Libreoffice Draw



LibreOffice
The Document Foundation

